

Postman API Automation Guide: Pre-request & Post-response Scripts

This guide covers the essentials for automating API workflows in Postman using **pre-request scripts**, **post-response scripts**, and chaining APIs effectively. It also touches on **JS basics in Postman**, **token generation**, and **correlation IDs**.

1. JavaScript Basics in Postman

Postman allows the use of JavaScript in scripts to manipulate requests and responses.

Common Use Cases:

- Dynamic variables
- Conditional request execution
- Generating timestamps, random values, or IDs
- Logging responses for debugging

Example:

```
// Generate a random user ID  
pm.environment.set("userId", Math.floor(Math.random() * 10000));
```

2. Pre-request Scripts

Pre-request scripts run **before a request is sent**. They are used to prepare data, generate tokens, or modify request parameters dynamically.

Common Tasks:

- Generating authentication tokens
- Setting environment variables
- Computing timestamps
- Setting correlation IDs

Example: Token Generation

```
// Example: JWT Token Generation
const jwt = require('jsonwebtoken'); // if using Node.js environment
const payload = { user: "testUser" };
const secret = "mySecretKey";

const token = jwt.sign(payload, secret, { expiresIn: '1h' });
pm.environment.set("authToken", token);
```

Example: Correlation ID

```
// Generate a unique correlation ID for tracking
const correlationId = "CID-" + Date.now();
pm.variables.set("correlationId", correlationId);
```

Use this `{{correlationId}}` in headers for request tracking.

3. Post-response Scripts

Post-response scripts run **after receiving the response**. They are useful for validating, extracting data, or chaining APIs.

Common Tasks:

- Extracting data from response
- Setting variables for next request

- Conditional assertions
- Logging response info for debugging

Example: Extracting a Value

```
// Extract userId from response JSON
const jsonData = pm.response.json();
pm.environment.set("createdUserId", jsonData.id);
```

4. Chaining APIs

Chaining APIs involves **using the output of one API as input for the next**. This is common for create → get → verify workflows.

Workflow Example:

1. **Create API**
 - Send request to create a resource
 - Extract the **id** from response using a post-response script
2. **Get API**
 - Use extracted **id** as path parameter to fetch the resource
3. **Verify API**
 - Assert values returned match expectations

Example:

```
// POST /create
const responseJson = pm.response.json();
pm.environment.set("resourceId", responseJson.id);
```

```
// GET /resource/:id
// Use {{resourceId}} in the request URL
```

```
// Verification
pm.test("Verify status is active", function () {
  const data = pm.response.json();
  pm.expect(data.status).to.eql("active");
});
```

5. Best Practices

- Keep pre-request and post-response scripts **small and reusable**.
- Use **environment and collection variables** instead of hardcoding.
- Always **log critical values** for debugging: `console.log(pm.variables.get("variableName"))`
- Use **correlation IDs** for tracking requests across APIs.
- Chain APIs to mimic **real-world workflows** and validate end-to-end functionality.